

AWS & Entra ID Federation — Complete Integration Patterns

Integrating Microsoft Entra ID with AWS authorization requires configuring OIDC federation, token validation, and claim mapping. This section provides configuration-ready code for the full identity-to-authorization pipeline.

Entra ID · ADFS · Application SAML · OAuth · Managed Identity · PrivateLink · VPC Endpoints

```
# Step 1: Entra ID Application Registration # (Configured in Azure Portal or via Microsoft Graph
API) Application Registration: Name: "Enterprise AI Authorization Platform" App Type: Web
Application Sign-in URL: https://api.bank.com/auth/callback API Permissions (Application
permissions): • Microsoft Graph: User.Read.All • Microsoft Graph: Group.Read.All • Microsoft
Graph: Directory.Read.All (Required for group GUID resolution in PIP Lambda) Expose an API:
Scope: api://bank-ai-platform/agent.invoke Scope: api://bank-ai-platform/tool.execute Scope:
api://bank-ai-platform/data.read Token Configuration → Optional Claims: Access Token: • groups
(Security groups as GUIDs, or names if < 200 groups) • upn • department • employee_id • country
• onprem_sid (for hybrid environments with ADFS) ID Token: • email • preferred_username App
Roles (for coarse-grained application roles): • Finance.Approver • Data.Reader • Agent.User •
Agent.Admin Conditional Access Policy: Target: This application Conditions: Require MFA for all
users Controls: Require compliant device OR hybrid Azure AD join
```

1.2 OIDC Federation Configuration

```
# Lambda Authorizer OIDC/JWT validation configuration # Uses AWS Lambda PowerTools for JWT
validation from aws_lambda_powertools.utilities.data_classes import APIGatewayProxyEvent from
aws_lambda_powertools.utilities.validation import validator import jwt import requests from
functools import lru_cache import json, time ENTRA_TENANT_ID = "YOUR-TENANT-ID" ENTRA_APP_ID =
"YOUR-APP-CLIENT-ID" # OIDC Discovery DISCOVERY_URL =
f"https://login.microsoftonline.com/{ENTRA_TENANT_ID}/v2.0/.well-known/openid-configuration" #
ADFS OIDC Discovery (for legacy federation) ADFS_DISCOVERY_URL =
"https://adfs.bank.com/adfs/.well-known/openid-configuration" @lru_cache(maxsize=1) def
get_jwks(issuer: str = 'entra') -> dict: # Cache JWKS for 1 hour (keyed by function instance
lifetime) if issuer == 'entra': discovery = requests.get(DISCOVERY_URL, timeout=5).json() else:
discovery = requests.get(ADFS_DISCOVERY_URL, timeout=5).json() jwks_uri = discovery['jwks_uri']
return requests.get(jwks_uri, timeout=5).json() def validate_token(token: str) -> dict: #
Decode header without verification to determine issuer header =
jwt.get_unverified_header(token) unverified = jwt.decode(token, options={'verify_signature':
False}) issuer = unverified.get('iss', '') # Route to correct JWKS based on issuer if
'login.microsoftonline.com' in issuer or 'sts.windows.net' in issuer: jwks = get_jwks('entra')
valid_audiences = [ENTRA_APP_ID, f'api://{ENTRA_APP_ID}'] valid_issuers = [
f'https://login.microsoftonline.com/{ENTRA_TENANT_ID}/v2.0',
f'https://sts.windows.net/{ENTRA_TENANT_ID}/' ] elif 'adfs.bank.com' in issuer: jwks =
get_jwks('adfs') valid_audiences = ['https://api.bank.com'] valid_issuers =
['https://adfs.bank.com/adfs'] else: raise ValueError(f"Unknown issuer: {issuer}") # Get
signing key from JWKS public_key = get_public_key_from_jwks(jwks, header['kid']) # Verify
signature, expiry, audience, issuer claims = jwt.decode( token, public_key,
algorithms=['RS256', 'ES256'], audience=valid_audiences, issuer=valid_issuers, options={
'verify_exp': True, 'verify_nbf': True, 'verify_iat': True, 'require': ['exp', 'iat', 'sub',
'iss', 'aud'] } ) # Validate jti (JWT ID) for replay prevention
validate_jti_not_replayed(claims.get('jti', ''), claims['exp']) return claims def
validate_jti_not_replayed(jti: str, exp: int): # Store jti in ElasticCache for token lifetime #
If already exists → replay attack detected import redis r =
redis.Redis(host='elasticsearch-endpoint') ttl = exp - int(time.time()) result =
r.set(f'jti:{jti}', '1', ex=ttl, nx=True) if not result: raise ValueError(f"JWT replay
```

```
detected: {jti})
```

1.3 ADFS Federation Deep Dive

Legacy ADFS environments require additional configuration because ADFS tokens use different claim schemas, group references, and token lifetimes than Entra ID:

Configuration Item	Entra ID	ADFS 4.0/5.0	Normalization Action
User identifier	sub (GUID), upn	upn, samaccountname	Normalize to upn as primary key
Groups	groups (GUID array)	group (DN strings or SAM names)	Parse DN or resolve SAM to capability
Token lifetime	Default: 1 hour (configurable)	Default: 8 hours (configurable)	Enforce max 1 hour at normalization layer
Audience	App client ID GUID	Relying Party URI	Map both to BANK_AI_PLATFORM constant
MFA claim	amr: ['mfa', 'pwd', 'rsa']	authmethodsreferences	Normalize to mfa_verified boolean
Department	department (direct claim)	department (AD attribute — may not be in token)	SCIM lookup if not in token
Custom attributes	extension_* prefix	Custom claim rules in ADFS	Both normalized to canonical snake_case
Token signing	RS256, ES256 (tenant JWKS)	RS256 (ADFS federation metadata)	Both validated via JWKS; separate endpoints

2. OAuth 2.0 On-Behalf-Of and Token Exchange

When an agent must call downstream enterprise APIs (SAP, Salesforce, internal microservices) on behalf of the user, it needs a token scoped to those APIs that carries the user's identity. OAuth 2.0 On-Behalf-Of (OBO) flow and RFC 8693 Token Exchange are the standards for this.

2.1 On-Behalf-Of Flow for Agent → Enterprise API

USER AUTHENTICATES • Receives access_token (scoped to AI Platform: api://bank-ai-platform) ■ ▼ Agent invokes with user's token AGENT RUNTIME (ECS Fargate) ■ ■ On-Behalf-Of Request to Entra ID: ■ POST https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token ■ grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer ■ client_id={agent-service-client-id} ■ client_secret={from Secrets Manager} ■ assertion={user-access-token} ■ requested_token_use=on_behalf_of ■ scope=https://api.sap.bank.com/.default ■ ▼ Entra validates: ■ • Agent service has OBO permission for SAP API ■ • User has consented to agent acting on their behalf ■ • User is not blocked from SAP API ■ ▼ Returns: New access_token ■ • sub = user's identity (preserved) ■ • aud = SAP API ■ • scp = SAP.ReadWrite (from user's entitlement) ■ • act claim: { "sub": "agent-service-id" } ← RFC 8693 SAP API CALL • Validates OBO token from Entra • sub = user → logs as user action • act = agent → audit trail shows agent acted on behalf of user CEDAR AUTHORIZATION (on the agent side, before the OBO call): • Verify agent is authorized to call SAP API on behalf of this user • Verify user has required capability: can_access_sap • Cedar policy checked BEFORE OBO token is requested

2.2 Token Exchange Implementation

```
# RFC 8693 Token Exchange + OBO implementation
import boto3
import requests
import json
import secrets

boto3_client = boto3.client('secretsmanager')
avp = boto3.client('verifiedpermissions')
TENANT_ID = "YOUR-TENANT-ID"
TOKEN_ENDPOINT = f"https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/token"

def exchange_token_for_api(
    user_token: str,
    target_api: str,
    canonical_claims: dict,
    agent_id: str
) -> str:
    # Step 1: Cedar pre-authorization check
    # Can this agent call this API on behalf of this user?
    authz = avp.is_authorized(
        policy_store_id=POLICY_STORE_ID,
        principal={'entity_type': 'BankAI::Agent', 'entity_id': agent_id},
        action={'action_type': 'BankAI::Action', 'action_id': 'ExchangeTokenFor'},
        resource={'entity_type': 'BankAI::APITarget', 'entity_id': target_api},
        context={'context_map': {
            'delegated_user_id': {'string': canonical_claims['principal']['id']},
            'target_api_capability': {'string': f'can_access_{target_api.lower()}'},
            'user_has_capability': {'boolean': f'can_access_{target_api.lower()}' in canonical_claims['capabilities']}
        }}
    )
    if authz['decision'] != 'ALLOW':
        raise PermissionError(f"Agent not authorized to exchange token for {target_api}")

    # Step 2: Get agent service credentials from Secrets Manager
    secret = secrets.get_secret_value(
        SecretId=f'entra-agent-credentials/{target_api}'
    )
    agent_creds = json.loads(secret['SecretString'])

    # Step 3: Perform OBO token exchange
    response = requests.post(
        TOKEN_ENDPOINT,
        data={
            'grant_type': 'urn:ietf:params:oauth:grant-type:jwt-bearer',
            'client_id': agent_creds['client_id'],
            'client_secret': agent_creds['client_secret'],
            'assertion': user_token,
            'requested_token_use': 'on_behalf_of',
            'scope': f'https://{target_api}/.default'
        },
        timeout=10
    )
    if response.status_code != 200:
        raise RuntimeError(f"OBO exchange failed: {response.json()}")

    obo_token = response.json()['access_token']

    # Step 4: Audit log the token exchange
    log_token_exchange(
        agent_id,
        canonical_claims['principal']['id'],
        target_api
    )

    return obo_token
```

3. Network Security: PrivateLink & VPC Endpoints

All authorization traffic must traverse AWS's private network, never the public internet. VPC endpoints for AVP, DynamoDB, ElastiCache, and S3 ensure that authorization calls never leave the AWS backbone.

3.1 Required VPC Endpoints for Authorization Architecture

Service	Endpoint Type	Purpose	DNS Resolution
Amazon Verified Permissions	Interface endpoint (PrivateLink)	Cedar AVP IsAuthorized API calls — no internet egress	verifiedpermissions.us-east-1.amazonaws.com → private IP
DynamoDB	Gateway endpoint	PIP attribute lookups, audit log writes	dynamodb.us-east-1.amazonaws.com → private
S3	Gateway endpoint	OPA bundle downloads, audit log archive	s3.amazonaws.com → private
Secrets Manager	Interface endpoint	Agent credential retrieval	secretsmanager.us-east-1.amazonaws.com → private
AWS STS	Interface endpoint	Token exchange, assume role for OBO	sts.amazonaws.com → private
ElastiCache	Not applicable (VPC-native)	Claims cache (Redis) — already in VPC	Private IP within VPC subnet
CloudWatch Logs	Interface endpoint	Audit log delivery without NAT	logs.us-east-1.amazonaws.com → private
EventBridge	Interface endpoint	Authorization event publishing	events.us-east-1.amazonaws.com → private

3.2 Security Group Configuration

```
# Security Group Rules for Authorization Components #
Auth-SG (Lambda Authorizer, Claims Normalization ECS)
auth_sg_inbound: - port: 443 from: api-gateway-sg description: "API GW to auth"
- port: 443 from: agent-runtime-sg description: "Agents to auth"
auth_sg_outbound: - port: 443 to: vpc-endpoint-sg description: "To AVP/Secrets/STS"
- port: 6379 to: redis-sg description: "To ElastiCache Redis"
- port: 443 to: dynamodb-endpoint description: "To DynamoDB VPC endpoint"
- port: 443 to: 0.0.0.0/0 description: "To Entra ID JWKS (internet via NAT)" #
Redis-SG (ElastiCache)
redis_sg_inbound: - port: 6379 from: auth-sg description: "Auth service to Redis"
- port: 6379 from: pip-lambda-sg description: "PIP Lambda to Redis"
redis_sg_outbound: [] # Redis does not initiate connections #
Agent-Runtime-SG (ECS/EKS agent tasks)
agent_sg_inbound: - port: 443 from: mcp-gateway-sg description: "MCP responses"
agent_sg_outbound: - port: 443 to: auth-sg description: "To authorization layer"
- port: 443 to: mcp-gateway-sg description: "To MCP PEP gateway"
- port: 443 to: vpc-endpoint-sg description: "To Bedrock, AVP" #
Network ACL (defense in depth): # Deny all traffic not from authorized source SGs #
Explicit deny for RFC 1918 cross-subnet except auth paths
```

4. AWS IAM Identity Center Integration

AWS IAM Identity Center (formerly AWS SSO) bridges Entra ID to AWS IAM permissions. For Agentic AI, it provides the AWS-side identity for agent task roles while Entra ID provides the business identity. These are two parallel identity planes that must both be properly configured.

4.1 Dual Identity Plane Architecture

```
MICROSOFT ENTRA ID AWS IAM IDENTITY CENTER (Business Identity Plane) (AWS Resource Identity Plane)
User: john.smith@bank.com
Permission Set: AgentRuntimeUser
JWT: { capabilities, dept, mfa }
IAM Role: AgentRuntimeRole-FINANCE
↓ ↓ Claims Normalization
ECS Task Role (Canonical claims object) (AWS service permissions)
↓ ↓ CEDAR AUTHORIZATION IAM AUTHORIZATION (Business decisions) (AWS service access)
"Can john invoke PaymentTool?" "Can task role call Bedrock?"
RELATIONSHIP:
• A single ECS task has BOTH an IAM task role AND carries john's claims
• Cedar uses canonical claims for business authorization
• IAM task role controls which AWS APIs the task can call
• They are INDEPENDENT – Cedar cannot see IAM roles, IAM cannot see Cedar claims
• BOTH must be correct for an action to succeed
EXAMPLE: Task wants to call Bedrock + invoke PaymentTool:
1. IAM checks: does AgentRuntimeRole have bedrock:InvokeModel? → Yes (IAM)
2. Cedar checks: does john.smith have can_approve_payment? → Yes (Cedar)
3. BOTH must allow → action proceeds
```

4.2 IAM Identity Center SCIM Sync

```
# IAM Identity Center SCIM sync from Entra ID # Configured in AWS IAM Identity Center console: #
Settings → Identity source → External Identity Provider # SCIM endpoint:
https://scim.sso.region.amazonaws.com/instances/{id}/scim/v2/ # SCIM token: generated in IAM
Identity Center # SCIM sync provides: #
• Users synced from Entra ID → IAM Identity Center #
• Groups synced from Entra ID → IAM Identity Center #
• Groups mapped to Permission Sets → IAM Roles #
IMPORTANT: This controls AWS service access (IAM plane) # NOT business authorization (Cedar plane)
# Do NOT try to use IAM Identity Center groups for Cedar policies #
Recommended Permission Set mapping for Agent architecture:
Permission Sets: AgentRuntimeUser: IAM policies:
- bedrock:InvokeModel (specific model ARNs only)
- bedrock:InvokeAgent (specific agent ARNs only)
- verifiedpermissions:IsAuthorized (AVP read – for Lambda Authorizer)
- dynamodb:GetItem, Query (audit table)
- secretsmanager:GetSecretValue (with resource tag condition)
Condition: RequestedRegion = eu-west-1, us-east-1
AuthorizationServiceUser: IAM policies:
- verifiedpermissions:IsAuthorized
- verifiedpermissions:IsAuthorizedWithToken
- elasticache:Connect (for Redis claims cache)
- dynamodb:* (audit tables only, by tag condition)
PolicyAdminUser: IAM policies:
- verifiedpermissions:PutPolicy
- verifiedpermissions>DeletePolicy
- verifiedpermissions:UpdatePolicy
Condition: MultiFactorAuthPresent = true (MFA required for policy changes)
```


6. Authorization Disaster Recovery

The authorization layer is critical infrastructure. A failure in the PDP must not bring down the entire platform. The following DR patterns ensure continued operation while maintaining the security posture.

6.1 Authorization DR Scenarios

Failure Scenario	Impact	Response	Recovery Time Target
AVP regional outage	Cedar evaluation unavailable	Fail-closed: deny all non-cached requests. Cached decisions (300s TTL) continue serving cached traffic.	Automatic — no manual intervention. RTO: 0 (cached) to 5 min (cache expiry)
ElastiCache failure	Claims normalization falls to cold path (~30ms)	Auto-failover to replica. If both fail: Lambda Authorizer calls LDAP directly.	Automatic ElastiCache failover: ~30s
Claims Normalization ECS down	Cannot normalize new JWT tokens	Lambda Authorizer retries 3x, then fail-closed. Existing cached claims still work.	ECS service auto-recovery: ~2 min. Full RTO with cache: 0
Entra ID JWKS unavailable	Cannot validate new JWT signatures	JWKS cached in Lambda memory (1 hour). Serve from cache. Alert at 45 min.	Cache serves for 1 hour. MTTR target: 30 min
DynamoDB PIP table failure	Cannot enrich claims with SCIM attributes	Fail to cached attributes. If no cache: use minimal claims from JWT only (limited policies apply).	DynamoDB multi-AZ: automatic. Single-AZ: 1-5 min failover
Policy store corruption	Policies return unexpected decisions	Automated drift detection triggers. Rollback to last known-good Git SHA via pipeline.	Pipeline rollback RTO: 10-15 min

6.2 Multi-Region Authorization

```
# Multi-region AVP policy synchronization # (Required for global deployments or region
failover) # Pattern: Primary-Active / Secondary-Active # Policy store MUST be consistent across
regions # Option 1: AVP Policy Replication (AWS managed replication) # → Not yet supported
natively – use custom replication # Option 2: GitOps-driven multi-region deployment
(RECOMMENDED) # CI/CD pipeline deploys to all regions in sequence: deploy_pipeline: stages: -
eu-west-1-staging # Staging: EU West - us-east-1-staging # Staging: US East - eu-west-1-prod #
Prod: EU West (primary) - approval-gate # Human approval required - us-east-1-prod # Prod: US
East - ap-southeast-1-prod # Prod: AP Southeast # Each region has its own AVP policy store # All
stores share the same policy definitions from Git # Regional Lambda Authorizers use their local
AVP store # Consistency check (runs every 5 minutes via EventBridge): def
check_cross_region_policy_consistency(): regions = ['eu-west-1', 'us-east-1', 'ap-southeast-1']
policy_hashes = {} for region in regions: avp = boto3.client('verifiedpermissions',
region_name=region) policies = list_all_policies(avp, POLICY_STORE_IDS[region])
policy_hashes[region] = compute_canonical_hash(policies) reference = policy_hashes['eu-west-1']
for region, hash_val in policy_hashes.items(): if hash_val != reference:
trigger_alarm(f"POLICY_INCONSISTENCY: {region} differs from primary")
```